

SOCIAL NETWORK ANALYSIS for DATA SCIENTISTS

today's menu: GRAPH MACHINE LEARNING (LECTURE Week 11)

Your lecturer: Gergo Bocsardi

Playdate: 2026-11-09

Menu for today

SNA4DS

1. From Statistics to Machine Learning
2. Embeddings (the bread and butter)
3. **BREAK**
4. Graph Neural Networks (GNNs)
5. Running GraphML models

1. From Statistics to Machine Learning

So far it was statistics, now it's ML

SNA4DS

Statistical Science
2001, Vol. 16, No. 3, 199–231

Statistical Modeling: The Two Cultures

Leo Breiman

Abstract. There are two cultures in the use of statistical modeling to reach conclusions from data. One assumes that the data are generated by a given stochastic data model. The other uses algorithmic models and treats the data mechanism as unknown. The statistical community has been committed to the almost exclusive use of data models. This commitment has led to irrelevant theory, questionable conclusions, and has kept statisticians from working on a large range of interesting current problems. Algorithmic modeling, both in theory and practice, has developed rapidly in fields outside statistics. It can be used both on large complex data sets and as a more accurate and informative alternative to data modeling on smaller data sets. If our goal as a field is to use data to solve problems, then we need to move away from exclusive dependence on data models and adopt a more diverse set of tools.

1. INTRODUCTION

The values of the parameters are estimated from

The "data model" tradition: building models from theory

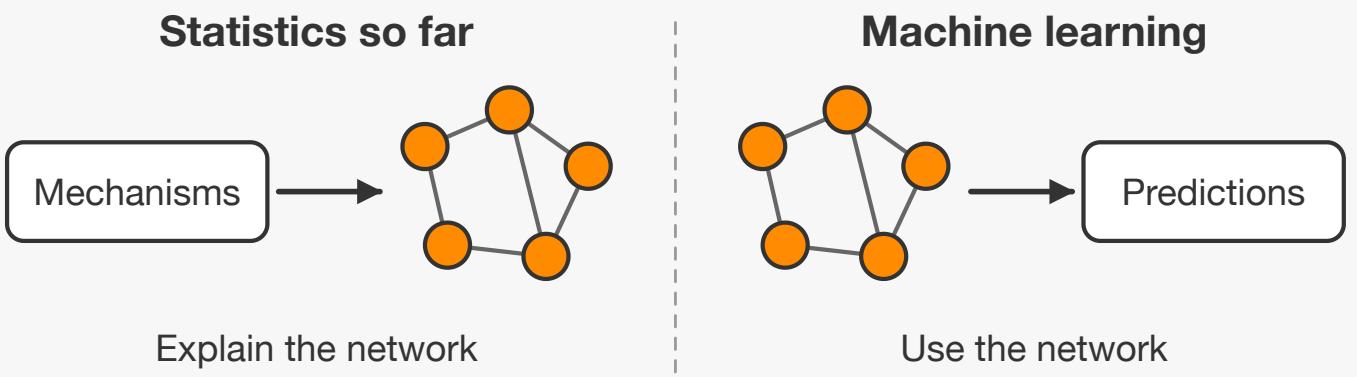
A lot of methods in this course come from the "data model" tradition:

- Understanding whether a network structure is random (CUG tests)
- Understanding what is the generative process (ERGM)

Machine Learning is different

ML is concerned with the other end

It takes the network as a given, and sees how we can use it for different tasks

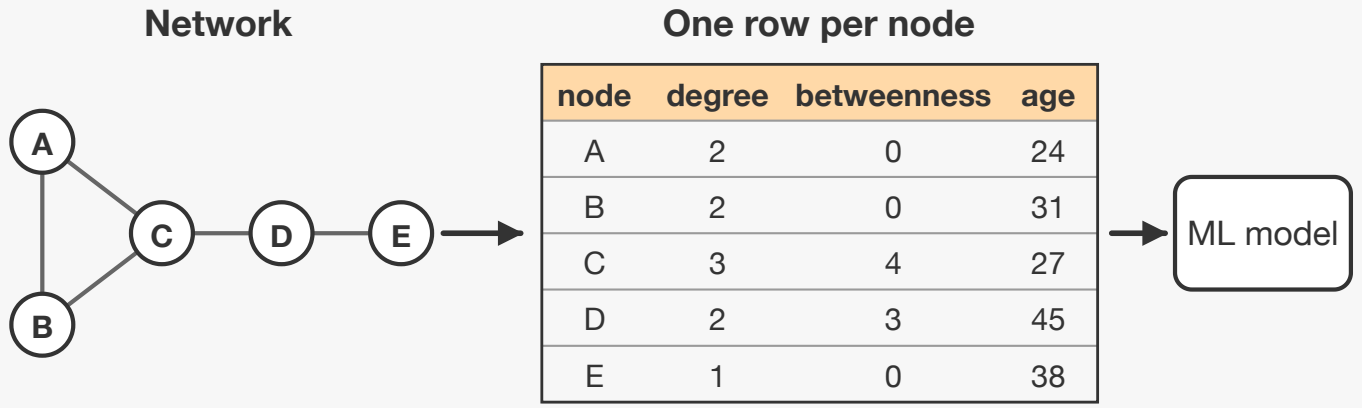


You can do "naive" ML

Use node features, centrality measures as inputs to ML models

(In ML models you're allowed to use centrality as predictors, in regressions you're not)

But... you're not really using the graph structure itself



2. Embeddings (the bread and butter)

To "use" a graph for tasks you need embeddings

To "use" a graph for tasks you need to turn it into a shape that algorithms can eat

This shape is called: EMBEDDINGS

This is going to be our bread and butter for today

What is an embedding?

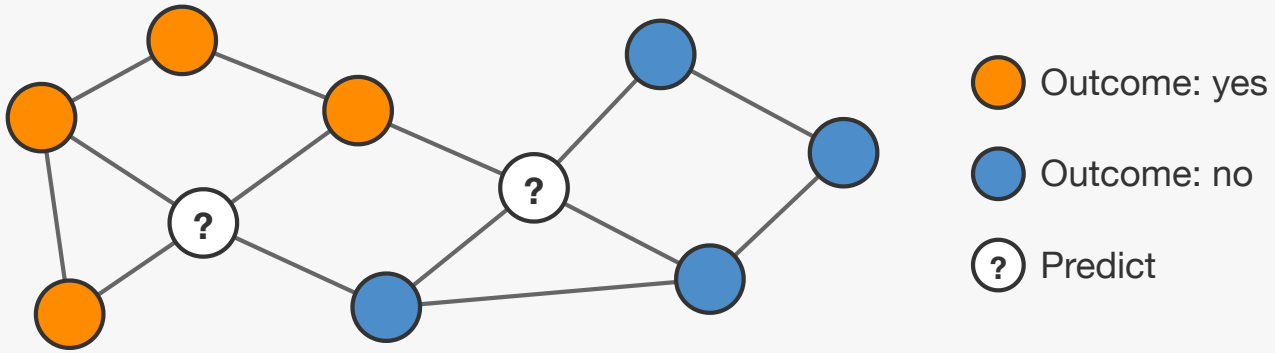
Further reading for the curious:

https://www.youtube.com/playlist?list=PLoROMvodv4rPLKxIpqhjhPgdQy7imNkDn

Most ML tasks are inherently node-level

Outcomes of people given their social graph, performance of firms given their allies, etc

This is exactly what node-level tasks are about!



Embeddings turn nodes into vectors

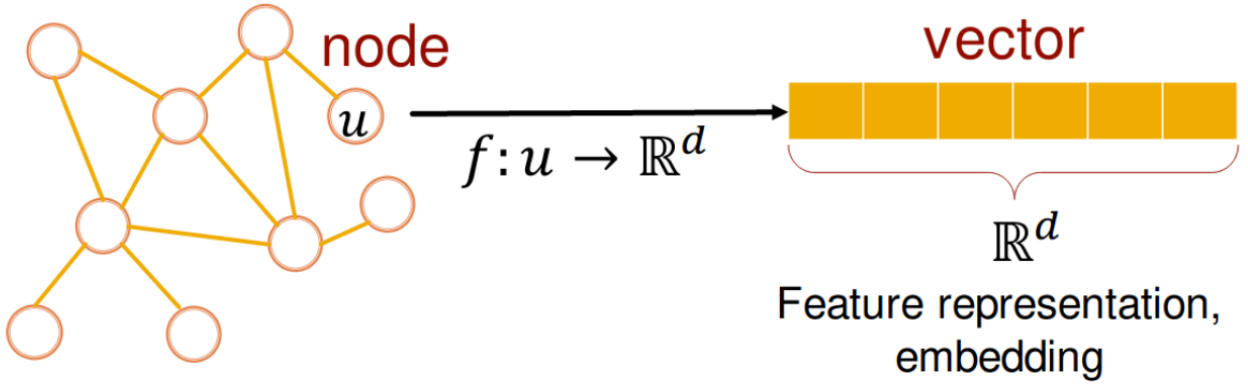


Figure 2.1: Node Level Embeddings of graphs map each node to a vector

Embeddings turn a node (or graph) into a vector

Then we can feed this vector to any algorithm we like!

The goal of embeddings

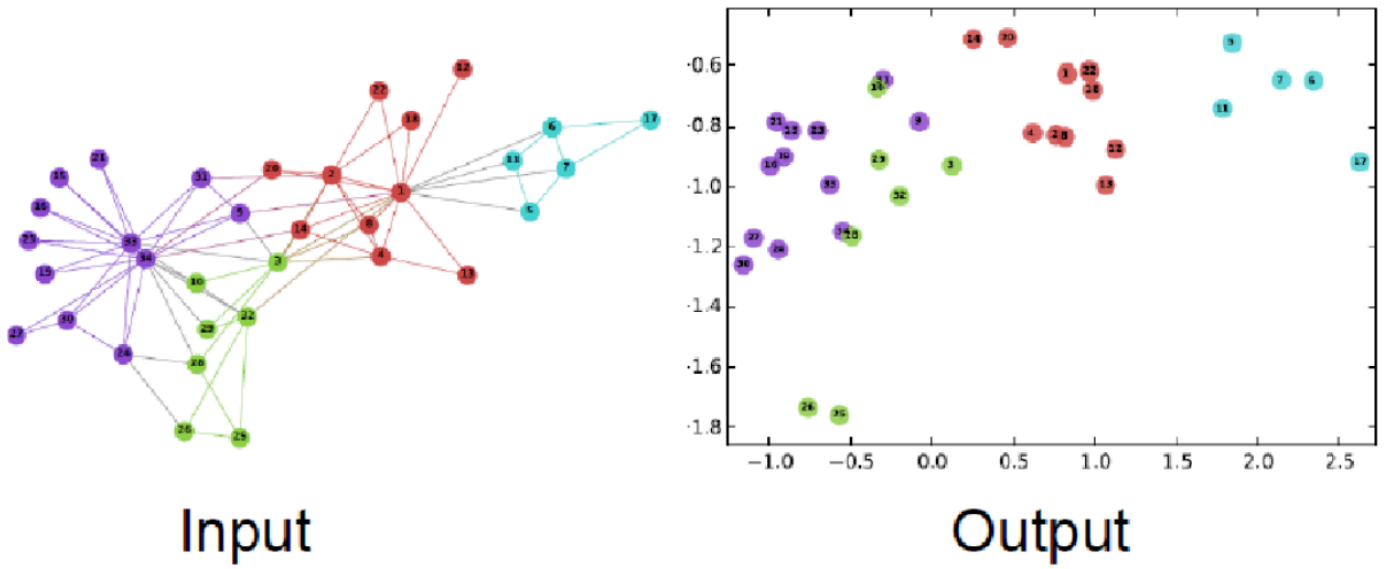


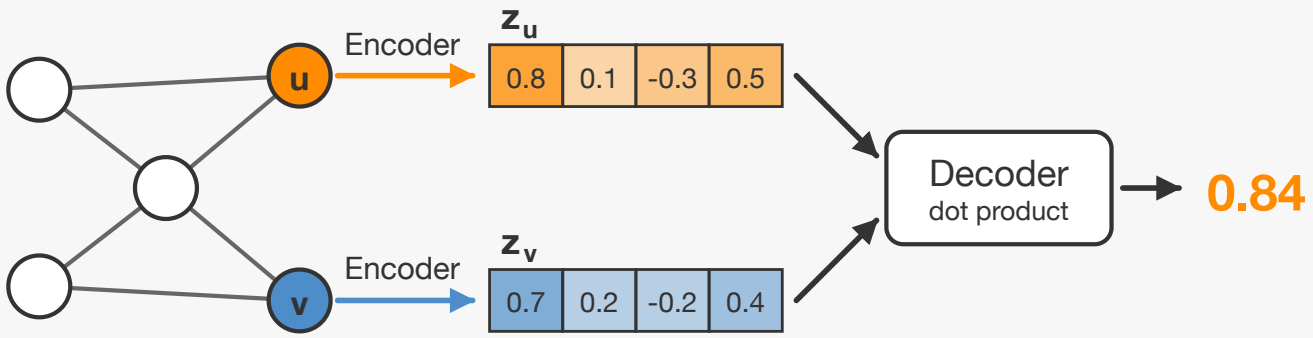
Figure 2.2: Embeddings of Karate Club Graph

Goal: similar position in the network = similar embedding vector

Embeddings at a high level

You need 3 things:

1. **Encoder**: looks up vector for any given node
2. **Decoder**: measures the similarity between two vectors
3. **Similarity definition**: what does "similar" mean? (distance in graph, topology, neighbors, etc.)



Train the encoder so the score matches similarity in the graph

The embedding framework

Similarity(u,v) = Decoder(Encoder(u), Encoder(v))

Of these three components, we learn the encoder when training

Different ways to make embeddings

SNA4DS

There are multiple approaches to creating embeddings

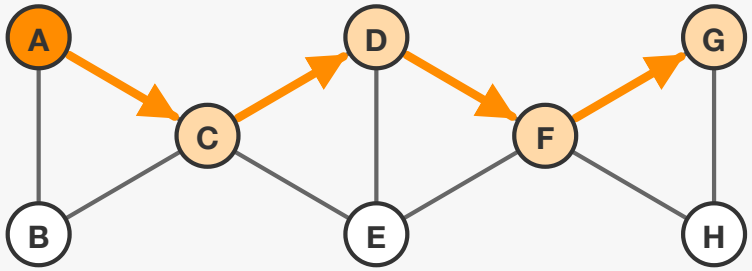
Let's look at some key methods:

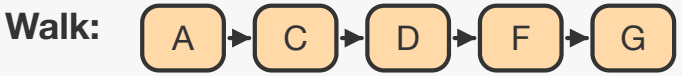
- Random walk
- DeepWalk
- Node2Vec

Random walk embeddings

Similarity(u,v) = P(u and v co-occur on a random walk over G)

The similarity between two nodes is defined by how often they appear together on random walks through the graph





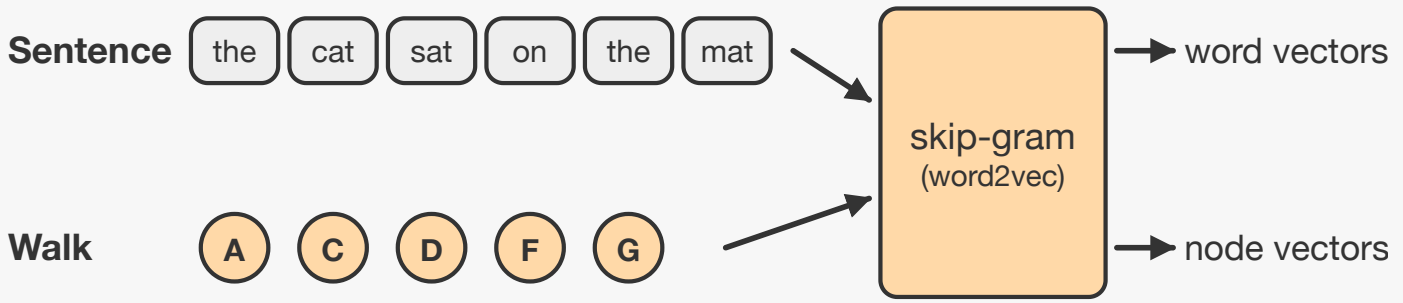
Nodes that often appear together on walks get similar embeddings

DeepWalk

Think word2vec, but for graphs!

- Walks are sentences
- Nodes are words
- Then we do skipgram next token prediction

You just do a bunch of walks, that is your corpus



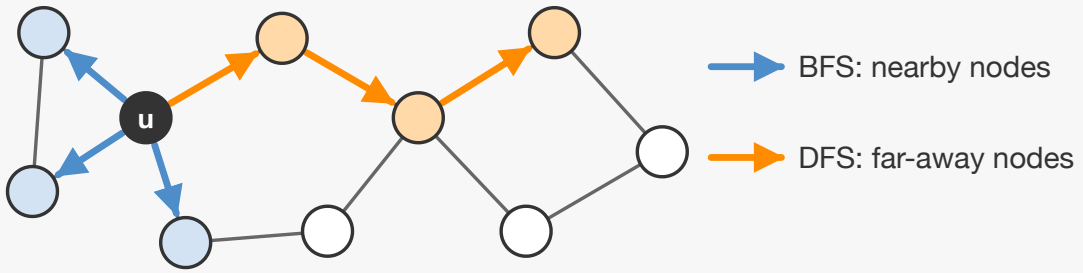
Node2Vec

Improved DeepWalk by adding biased walks

Not every path has an equal probability

You can tune how the walk-based exploration goes:

- BFS vs DFS
- Return parameter
- In-out parameter



BREAK

4. Graph Neural Networks (GNNs)

Not every task needs the same information (embeddings)

Different tasks require different kinds of information from a graph

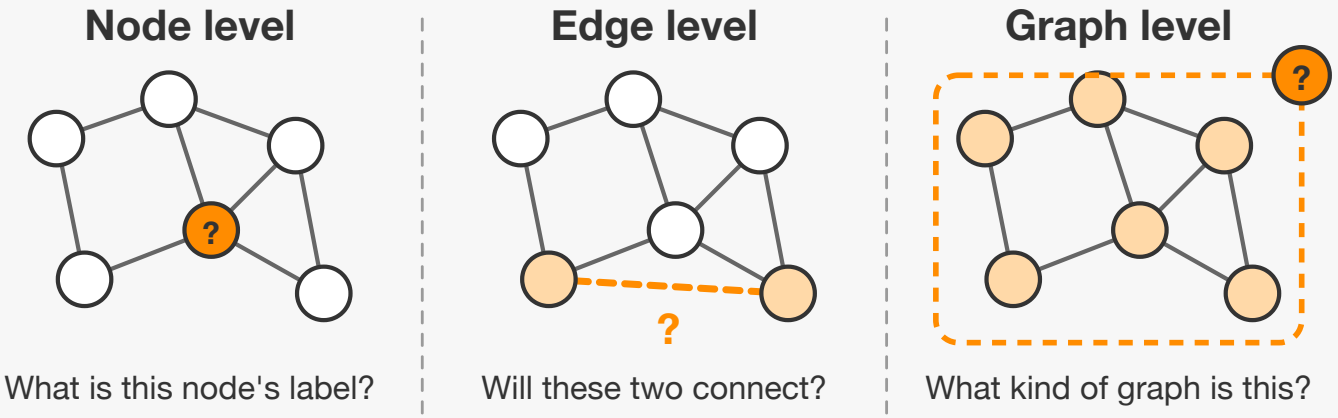
That's where GNNs come in

Task-specific embeddings, end to end

GNNs learn task-specific embeddings

Different kinds of tasks:

- Node level (classification/regression)
- Edge level (link prediction)
- Graph level (graph classification)



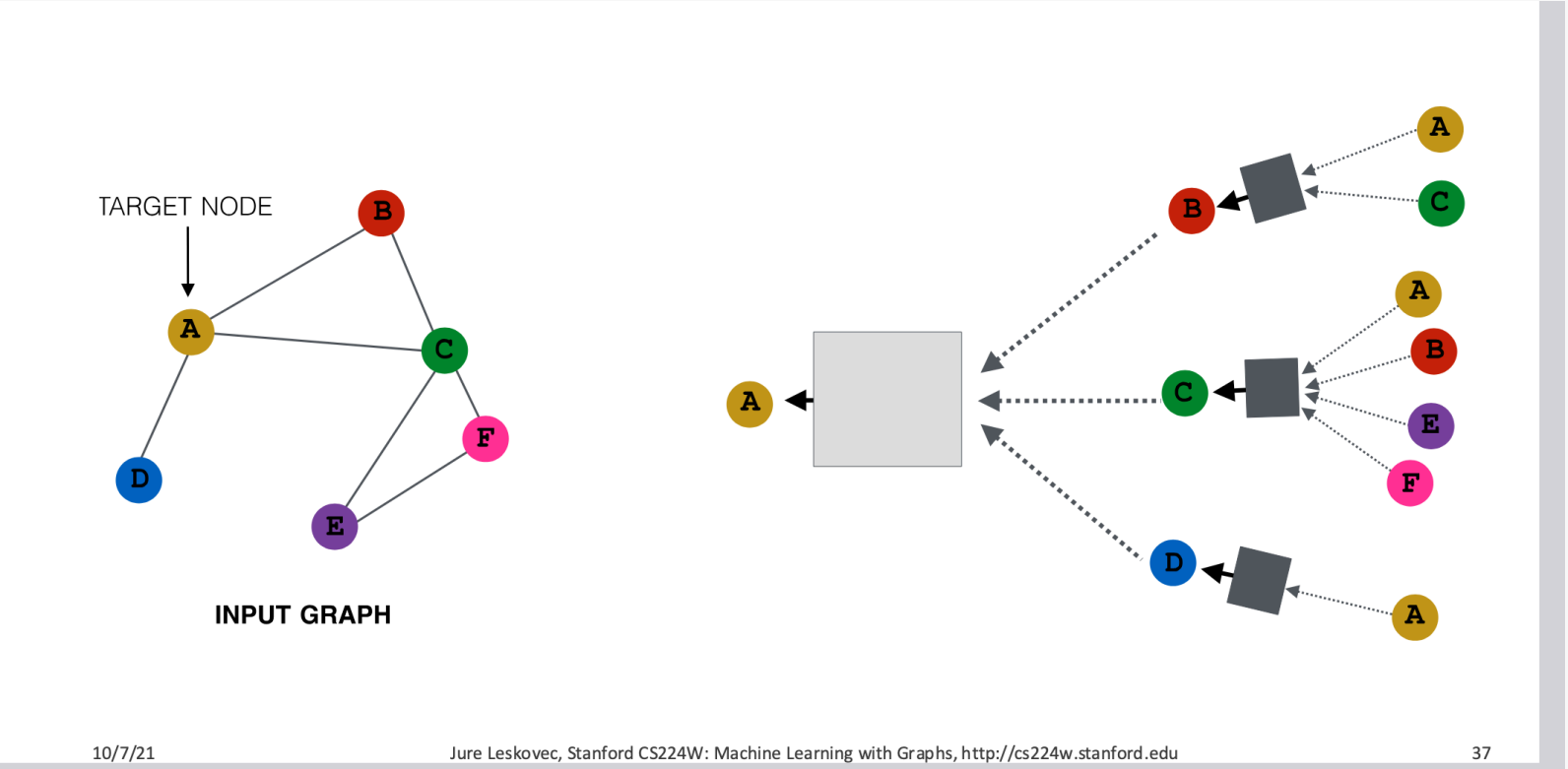
Graph Convolutional Networks (GCN)

SNA4DS

Key concepts:

- Compute graph
- Message passing
- Layers

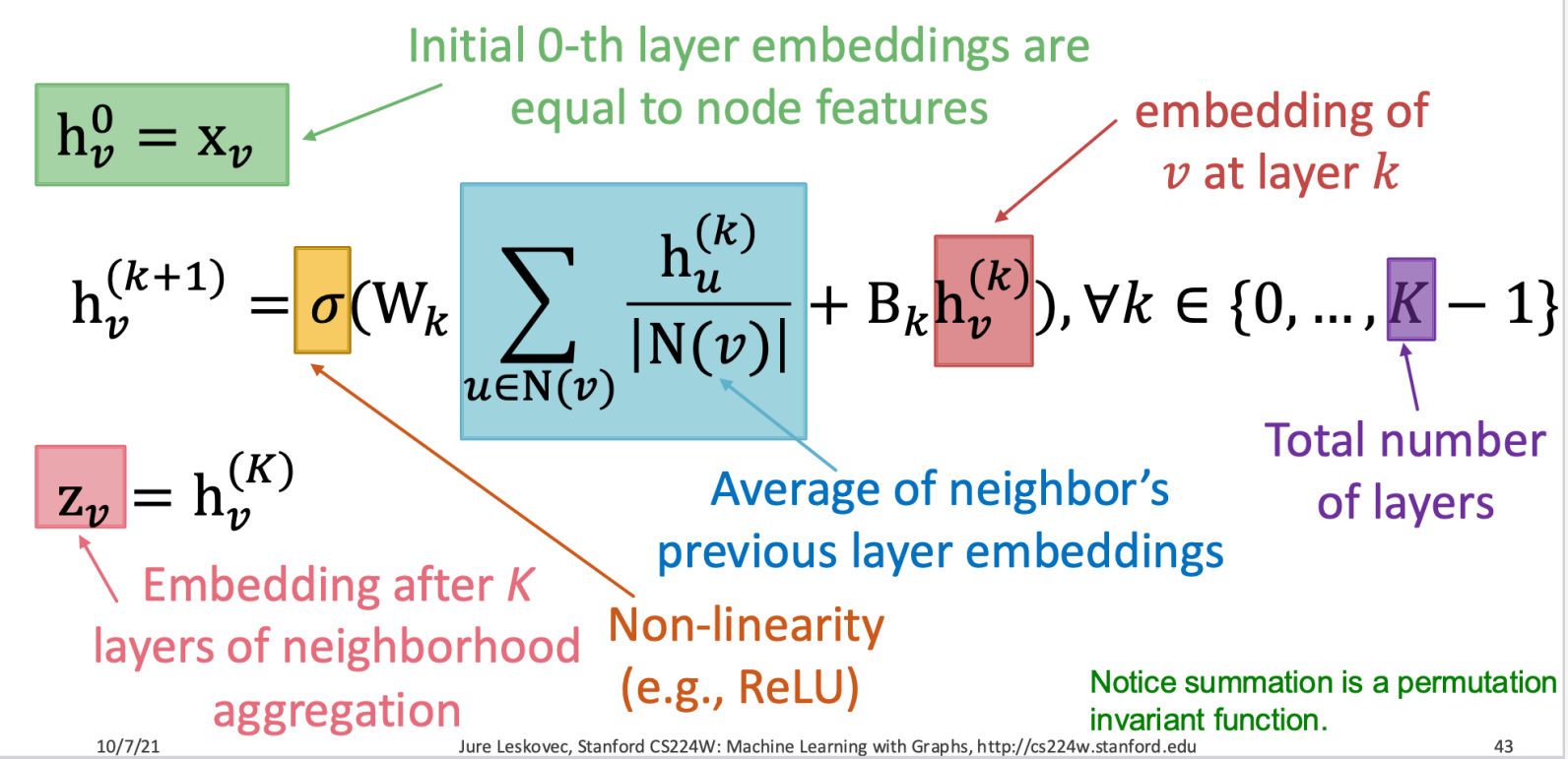
Compute Graphs in GCNs



This graph determines how we aggregate information from neighbors

From https://web.stanford.edu/class/cs224w/slides/03-GNN1.pdf

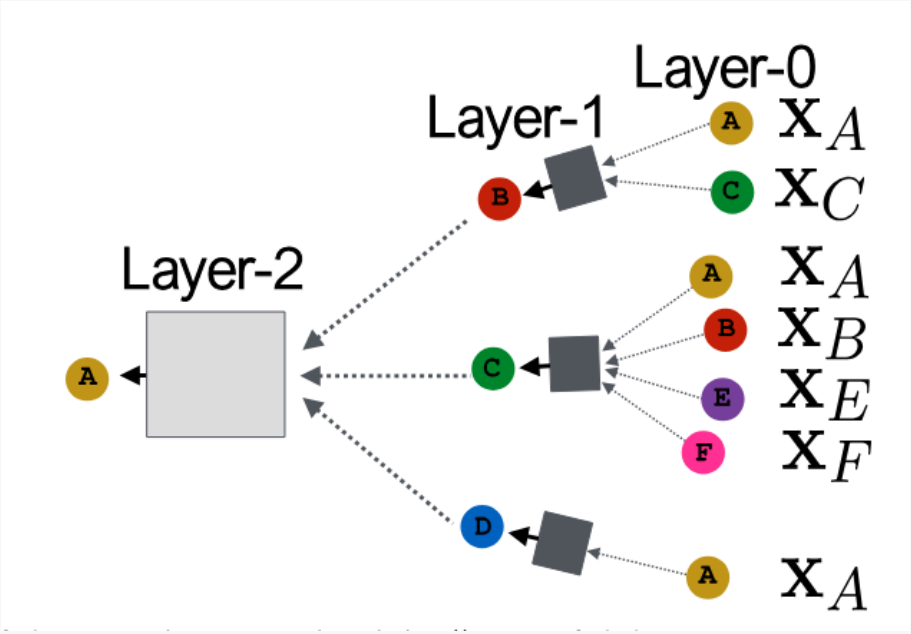
Message passing



You can just run backpropagation through all the aggregation neurons!

From https://web.stanford.edu/class/cs224w/slides/03-GNN1.pdf

GCN layers



In GNNs we refer to degrees of separation as "hops"

2 layers = 2-hop aggregation for each node

More layers = information from further away in the graph --> MORE ISN'T ALWAYS BETTER

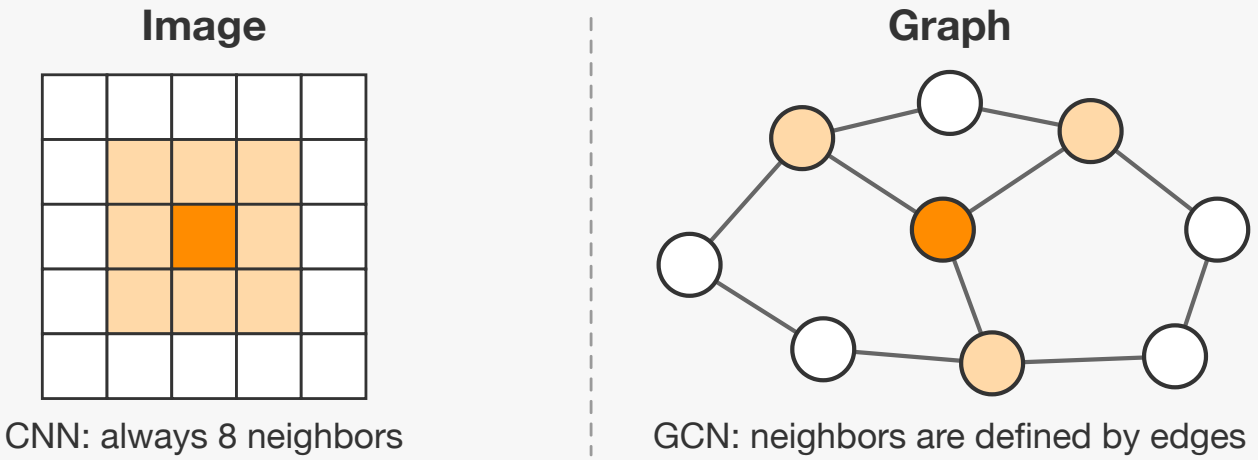
From https://web.stanford.edu/class/cs224w/slides/03-GNN1.pdf

GCNs are generalized CNNs

GCNs are basically more general CNNs

In CNNs: neighborhood is defined by pixels (every node has 8 neighbors)

In GCNs: neighborhood is defined by edges



Graph Attention Networks (GAT)

Weighted neighbor aggregation

Replace the average of neighbors' embeddings with a weighted average:

$$\underbrace{\sum_{u \in N(v)} \frac{\mathbf{h}_u^{(k)}}{|N(v)|}}_{\text{equal-weight average}} \longrightarrow \underbrace{\sum_{u \in N(v)} \alpha_{vu}^{(k)} \mathbf{h}_u^{(k)}}_{\text{learned weighted average}}$$

- $\alpha_{vu}^{(k)}$ is the weight given to neighbor u when updating node v at layer k.
- A neural network computes the weights from the embeddings of v and u.
- The weights are nonnegative and sum to 1 across neighbors.

Veličković et al. (2018), Graph Attention Networks.

And many more...

There are many more GNN architectures

But we don't have time to cover them all today

5. Running GraphML models

How to run them

PyTorch Geometric is the main library for GNNs



Great tutorials and examples available:

https://pytorch-geometric.readthedocs.io/en/latest/get_started/colabs.html

Thanks for the attention!

Let me know if you have any questions!